VECTOR DATABASES & CHROMADB

Explanatory study notes with practical examples, revision summary, brainstorming questions, and model answers

Learning outcomes

- Understand vectors, embeddings, similarity metrics, and approximate nearest-neighbor search.
- Distinguish dedicated vector databases, vector libraries, and traditional databases with vector-search extensions.
- Explain how ChromaDB supports retrieval workflows and build a simple similarity-search example.
- Apply the concepts to enterprise RAG, recommendation, image retrieval, and operational use cases.

Accuracy note: FAISS and Annoy are primarily vector-search libraries rather than full database vendors. MongoDB and Cassandra are products/projects, not “Apache MongoDB.” Inverted indexes are different from vector indexes, although hybrid systems may use both.

Contents

1	Foundations: vectors, embeddings, and vector databases
2	Similarity search and manual calculations
3	Architecture, indexing, and scaling
4	Database categories and technology landscape
5	Enterprise and AI use cases
6	ChromaDB concepts, deployment, and code
7	RAG workflow and production considerations
8	Quick revision summary
9	Brainstorming questions with model answers
10	Self-check quiz with answer key

1. Foundations: Vectors, Embeddings, and Vector Databases

What is a vector?

A vector is an ordered array of numerical values. Each dimension represents a feature, whether directly designed by a human or learned by a model. For example, [0.82, 0.15, 0.64] could encode hidden characteristics of a document, image, sound, or user profile.

What is an embedding?

An embedding is a vector created by a model so that items with related meaning or characteristics are located near one another in vector space. A text embedding model may place “ATM cash withdrawal problem” near “cash dispenser transaction failure,” even when the exact words differ.

What is a vector database?

A vector database stores vectors together with identifiers, source content, and metadata. Its key capability is retrieving the nearest vectors to a query vector. This enables meaning-based retrieval instead of relying only on exact keywords.

Traditional search vs. vector search

Keyword search finds lexical matches. Vector search finds semantic or feature similarity. Hybrid search combines both. In an enterprise knowledge assistant, keyword search may precisely match an incident ID, while vector search finds troubleshooting notes that describe the same symptom using different wording.

Supported data types

Text, images, audio, video frames, patterns, maps, biological sequences, and other complex data can be converted into vectors by an appropriate embedding or feature-extraction model. The database normally stores the vector plus a reference to the original object or text chunk.

2. Similarity Search and Manual Calculations

Basic workflow

1) Convert stored items into vectors. 2) Convert the query with the same embedding model. 3) compute distance or similarity. 4) rank candidates. 5) return the top-k results, optionally after metadata filtering or reranking.

Cosine similarity

Cosine similarity measures the angle between vectors: cos(A,B) = (A dot B) / (||A|| ||B||). A higher value generally means more similar direction. It is often useful when direction matters more than magnitude.

Euclidean distance

Euclidean distance measures straight-line distance: d(A,B) = sqrt(sum((Ai-Bi)^2)). A smaller distance means the vectors are closer.

Dot product

The dot product is sum(Ai x Bi). It can be fast and useful when the embedding model is trained for it. If vectors are normalized, dot product and cosine similarity produce equivalent ranking.

Manual example

Query Q=[1,1], A=[1,2], B=[2,3]. Euclidean distances are d(Q,A)=1 and d(Q,B)=sqrt(5)=2.236. Therefore A is nearer to Q. This simple calculation illustrates exact or brute-force nearest-neighbor search.

3. Architecture, Indexing, and Scaling

Exact search

Exact search compares the query with every candidate. It maximizes recall but becomes expensive as collections and vector dimensions grow.

Approximate nearest neighbor (ANN)

ANN indexes avoid scanning every vector. They return very close, but not always mathematically exact, neighbors much faster. The main engineering trade-off is recall versus latency, memory, and cost.

Graph-based indexes

Graph indexes such as HNSW connect nearby vectors. Search navigates the graph toward promising regions. They commonly provide strong retrieval quality and low latency but may require significant memory.

Locality-sensitive hashing (LSH)

LSH hashes similar vectors into the same or nearby buckets with high probability. A query searches likely buckets instead of the entire collection.

Product quantization (PQ)

PQ compresses vectors into compact codes. It reduces memory and can speed distance estimation, but compression may reduce retrieval accuracy.

Distributed computing and parallelism

Sharding spreads vectors across nodes, while replicas improve availability and read capacity. Parallel query execution searches partitions at the same time. Production design must also address consistency, rebalancing, backup, monitoring, and failure recovery.

4. Database Categories and Technology Landscape

In-memory and disk-based

In-memory systems emphasize speed but consume more RAM. Disk-oriented or memory-mapped approaches support larger collections at lower memory cost, usually with different latency characteristics.

Distributed systems

Distributed vector databases are designed to scale storage and query processing across nodes. They are suitable when a single host cannot meet capacity, throughput, or availability requirements.

Graph and time-aware workloads

Graph relationships and time-series data are separate data models, though some platforms combine them with vector search. Choose the primary store according to the dominant access pattern rather than assuming one vector database replaces all database types.

Libraries vs. databases

FAISS and Annoy are commonly used search libraries. Milvus is a distributed vector database. PostgreSQL with pgvector, Elasticsearch, Redis, MongoDB, MySQL variants, Cassandra-based platforms, and SingleStore can provide vector-search capabilities within broader database or search platforms.

Selection criteria

Evaluate scale, latency, recall, filtering, update frequency, durability, consistency, deployment model, security, observability, backup, ecosystem integration, and operational skill requirements.

5. Enterprise and AI Use Cases

Semantic document search

Embed document chunks and retrieve passages that are conceptually related to a user question. Example: searching “cassette marked fatal by switch” may retrieve notes discussing host-side cassette status even without an exact phrase match.

Recommendation systems

Represent users and items as vectors. Recommend items close to a user profile or to previously selected items. Cross-domain recommendations can connect related interests from different content categories when the embedding design supports it.

Image and video retrieval

Encode images or frames with a vision model. A query image or text description can retrieve visually or semantically related media. Metadata filters can restrict results by date, camera, location, or product category.

Geospatial and fleet workloads

Vectors can represent learned route, traffic, or location patterns, but conventional spatial indexes remain important for exact geographic operations. Hybrid architectures often combine geospatial databases with learned-vector retrieval.

Marketing and social analysis

Embeddings can support profile similarity, content recommendation, clustering, trend discovery, and anomaly detection. Privacy, consent, fairness, and explainability must be considered.

Classification and clustering

Nearest-neighbor methods can classify an item using nearby labeled examples. Clustering groups similar vectors without predefined labels. The quality depends heavily on the embedding model and data preparation.

6. ChromaDB Concepts, Deployment, and Code

What ChromaDB provides

ChromaDB is designed for embedding-based retrieval workflows. A collection groups records containing IDs, documents or payloads, embeddings, and metadata. Applications add records and query for similar items.

Retrieval capabilities

Typical workflows combine vector similarity with metadata filtering. Depending on configuration and integration, developers may also combine lexical or full-text retrieval with vector results for hybrid search.

Deployment modes

A local or persistent client is convenient for development and smaller applications. A client-server deployment separates applications from the database service and is more suitable when multiple clients require a shared service. Production readiness still requires deliberate security, backup, monitoring, and capacity design.

ANN behavior

ChromaDB uses ANN indexing to retrieve approximately closest records efficiently. “Approximate” means the system optimizes speed and scale while aiming for high recall rather than guaranteeing exhaustive comparison.

Framework and language integration

ChromaDB is commonly used with Python and JavaScript/TypeScript ecosystems and can be integrated into orchestration frameworks. Always check the current client and server compatibility for your selected version.

ChromaDB Similarity Search - Practical Example

The following example demonstrates a small local collection. In production, pin tested package versions, choose an approved embedding model, and configure persistence, authentication, logging, backup, and lifecycle controls.

```
# Install in an isolated environment
pip install chromadb

import chromadb

client = chromadb.PersistentClient(path="./chroma_data")
collection = client.get_or_create_collection(name="support_notes")

collection.add(
ids=["INC-001", "INC-002", "INC-003"],
documents=[
"ATM cassette status changed to fatal by host switch",
"Windows deployment failed after BIOS update",
"Cash withdrawal completed but journal parsing failed"
],
metadatas=[
{"bank": "OAB", "type": "ATM"},
{"bank": "Internal", "type": "Endpoint"},
{"bank": "OAB", "type": "Parser"}
]
)

results = collection.query(
query_texts=["Why did the switch mark the cassette fatal?"],
n_results=2,
where={"bank": "OAB"}
)

print(results["ids"])
print(results["documents"])
```

How it works

- The collection stores text, identifiers, and metadata.
- The client embeds the query and compares it with indexed document vectors.
- The metadata filter limits the search to OAB records before or during retrieval, depending on implementation.
- The returned records are ranked by similarity. The application should inspect distances/scores and apply business rules rather than assuming every top result is relevant.

Manual similarity-search pseudocode

```
query_vector = embed(query)
for each document_vector:
  score = cosine_similarity(query_vector, document_vector)
sort scores descending
return top_k
```

7. RAG Workflow and Production Considerations

1. Ingest	Read approved source documents and retain source identity, version, permissions, and timestamps.
2. Chunk	Split content into coherent passages. Chunk size and overlap affect recall, precision, context quality, and cost.
3. Embed	Generate vectors with one controlled embedding model. Changing the model generally requires re-embedding the collection.
4. Index	Store vectors, content references, metadata, and access-control attributes.
5. Retrieve	Embed the question, apply permission and metadata filters, retrieve top-k candidates, and optionally run hybrid search.
6. Rerank	Use a stronger model or deterministic rules to reorder candidates and remove weak matches.
7. Generate	Send only relevant, authorized context to the LLM with instructions to answer using the provided evidence.
8. Validate	Return citations, log retrieval diagnostics, evaluate groundedness, and capture feedback.

Production checklist

- Security: authentication, authorization, tenant isolation, encryption, secrets management, and audit logs.
- Data governance: ownership, classification, retention, deletion, lineage, and source permissions.
- Quality: retrieval recall, precision, relevance, groundedness, and regression test sets.
- Operations: health checks, capacity planning, backup/restore testing, index rebuild plans, monitoring, and alerting.
- Cost: embedding generation, storage, replicas, query throughput, reranking, and LLM token usage.
- Change management: model/version pinning, schema evolution, re-embedding strategy, and rollback.

8. Quick Revision Summary

Vector	An ordered list of numbers representing features.
Embedding	A learned vector representation designed to preserve useful similarity.
Vector database	A system that stores and retrieves vectors with IDs, payloads, and metadata.
Similarity search	Retrieval of vectors nearest to a query vector.
Cosine similarity	Compares vector direction; higher is usually more similar.
Euclidean distance	Measures straight-line distance; lower is more similar.
ANN	Fast approximate retrieval that trades some exactness for scale and latency.
HNSW	Graph-based ANN index with strong retrieval quality and low latency.
LSH	Hashing approach that makes similar vectors likely to share buckets.
PQ	Compression technique that reduces vector memory and accelerates approximate distance calculations.
Metadata filtering	Limits candidates using structured attributes such as bank, date, type, or permission.
Hybrid search	Combines lexical and vector retrieval.
ChromaDB	A vector store commonly used for embedding retrieval and RAG prototypes/applications.
RAG	Retrieve relevant evidence, then use it as context for generation.

9. Brainstorming Questions with Model Answers

Use these for discussion, interviews, architecture reviews, or written practice. The answers are models, not the only acceptable solutions.

Q. Why can vector search find relevant content when keywords do not match?

Model answer: Because both the stored content and query are mapped into an embedding space where semantic or feature similarity is represented by geometric proximity. The retrieval compares vectors rather than requiring identical words.

Q. When should an enterprise use hybrid search instead of vector-only search?

Model answer: When exact tokens matter, such as incident IDs, error codes, account numbers, product names, or policy clauses, while semantic matching is also useful. Lexical retrieval protects exactness; vector retrieval broadens recall.

Q. Why must queries and documents use compatible embedding models?

Model answer: Distances are meaningful only when vectors share the same learned space and dimensional interpretation. Mixing incompatible models can make similarity scores meaningless.

Q. What happens if the embedding model is changed?

Model answer: Existing vectors may no longer be comparable with new query vectors. A controlled migration normally requires re-embedding, rebuilding indexes, validating retrieval quality, and planning rollback.

Q. How would you choose between exact and ANN search?

Model answer: Use exact search for small collections, validation, or cases where perfect nearest-neighbor recall is essential. Use ANN when latency, throughput, or scale makes exhaustive comparison impractical.

Q. What is the main risk of returning top-k results without a threshold?

Model answer: The system may always return something even when nothing is sufficiently relevant. Use score/distance analysis, thresholds, reranking, or an explicit “insufficient evidence” outcome.

Q. How does chunk size affect RAG?

Model answer: Very small chunks may lose context; very large chunks may dilute relevance and consume tokens. Select chunking based on document structure and evaluate it with representative questions.

Q. Why is metadata important?

Model answer: Metadata supports security filters, tenant isolation, date and product filtering, source tracking, deletion, governance, and explainability. A vector without trustworthy metadata is difficult to manage operationally.

Q. How would you secure a bank knowledge-base vector store?

Model answer: Enforce source permissions during retrieval, isolate tenants, encrypt data, authenticate clients, authorize collections, protect secrets, audit access, classify content, apply retention, and test that users cannot retrieve unauthorized chunks.

Q. How can vector search help ATM incident troubleshooting?

Model answer: Embed historical incidents, symptoms, log summaries, resolutions, and component notes. A new symptom description can retrieve similar incidents. Engineers must still validate machine model, software version, date, configuration, and source evidence.

Q. Why might a semantically similar result still be wrong?

Model answer: Semantic closeness does not prove factual applicability. The result may concern a different product version, customer, period, policy, or root cause. Filters, reranking, citations, and human validation remain necessary.

Q. What metrics would you monitor?

Model answer: Retrieval recall and precision on a labeled test set, relevance at k, latency percentiles, error rate, index size, query throughput, ingestion lag, cost, permission-filter failures, and answer groundedness for RAG.

Q. How do PQ and HNSW differ conceptually?

Model answer: PQ compresses vectors and estimates distances using compact codes. HNSW organizes vectors in a navigable proximity graph. Systems may use them separately or in combination depending on scale and memory goals.

Q. Should large binary files be stored directly in a vector database?

Model answer: Usually store embeddings and metadata with a pointer to the governed source object. Keep original files in an appropriate object, document, or content store unless the chosen platform explicitly supports and operationally justifies direct payload storage.

Q. What is a practical deployment path for ChromaDB?

Model answer: Start locally with a small validated corpus and evaluation set. Then move to a controlled shared service only after defining persistence, authentication, network controls, backup, monitoring, capacity, version compatibility, and recovery procedures.

Q. How would you reduce hallucination in a RAG chatbot?

Model answer: Improve retrieval and chunking, apply access filters, rerank results, require citations, instruct the model to use supplied evidence, allow an insufficient-evidence response, and measure groundedness with regression tests.

Q. Can a vector database replace a relational database?

Model answer: Usually no. Relational databases remain strong for transactions, constraints, joins, and structured reporting. Vector retrieval is a specialized access pattern and often complements existing systems.

Q. What causes slow vector queries?

Model answer: Possible causes include an unsuitable index, high candidate counts, expensive filters, large dimensions, insufficient memory, poor sharding, network latency, concurrent load, index updates, or reranking overhead.

Q. How would you evaluate two vector stores fairly?

Model answer: Use the same dataset, embeddings, hardware class, filters, concurrency, and target recall. Compare latency percentiles, throughput, memory, build time, update behavior, availability, operations, security, and total cost.

Q. What would make you reject a vector result in production support?

Model answer: Insufficient score, wrong bank or machine type, incompatible software version, stale source, unauthorized content, missing provenance, contradictory evidence, or failure to match critical structured conditions.

10. Self-Check Quiz

1. Define an embedding in one sentence.

2. Which metric is higher for more similar normalized vectors: cosine similarity or Euclidean distance?

3. What does ANN trade for speed and scalability?

4. Name two reasons to use metadata filtering.

5. Why can vector-only search be weak for error codes?

6. What is the difference between FAISS and Milvus at a high level?

7. What should happen after changing the embedding model?

8. Name three production controls for a client-server vector database.

9. What is the purpose of reranking?

10. Why should RAG answers include citations?

Answer Key

1. An embedding is a numerical representation created so that useful similarity is reflected by proximity in vector space.
2. Cosine similarity is higher for more similar direction; Euclidean distance is lower for closer vectors.
3. ANN trades a degree of exact nearest-neighbor certainty or recall for lower latency and better scale.
4. Examples: enforce permissions, restrict by bank/product/date/type, improve relevance, and support governance.
5. Error codes require exact lexical matching; semantically oriented embeddings may dilute or tokenize them poorly.
6. FAISS is primarily a vector-search library; Milvus is a distributed vector database.
7. Re-embed affected content, rebuild indexes, test retrieval quality, migrate in a controlled manner, and keep rollback options.
8. Authentication/authorization, encryption, backup/restore, monitoring, network controls, auditing, and capacity planning.
9. Reranking applies a stronger relevance model or rules to reorder and filter initial candidates.
10. Citations let users verify evidence, improve transparency, and help identify unsupported answers.

Final takeaway

Vector databases make unstructured and multimodal information searchable by learned similarity. Their value comes not only from fast indexes, but from the complete engineering system: suitable embeddings, careful chunking, structured metadata, access control, evaluation, observability, and source-grounded application design.